

GCImpOpt: Learning efficient goal-conditioned policies by imitating optimal trajectories



Jon Goikoetxea^{1,2}

Jesús F. Palacián^{1,3}

¹Dept. of Statistics, Mathematics and Computer Science, Public University of Navarra

²Ensimag, INP Grenoble, Université Grenoble Alpes

³Institute for Advanced Materials and Mathematics (InaMat²)

Correspondence: Jon.Goikoetxea-Macua@etu.univ-grenoble-alpes.fr



Small NN policies trained on optimal trajectories achieve **fast, near-optimal goal-reaching control**.

1 Motivation

We want controllers that are:

- general, **(goal-conditioned)**
- efficient, **(near-optimal)**
- fast. **(low control latency)**

Model-Predictive Control:
computationally expensive.

Imitation learning:
demonstrations are suboptimal or expensive to collect.

Goal-Conditioned Reinforcement Learning:
online exploration is costly.

2 Problem setting

Known system model

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t))$$

where we aim to reach any goal $\mathbf{g}$ starting from any state $\mathbf{x}_0$

$$\mathbf{x}_0 = \mathbf{x}(0) \quad \mathbf{g} = \mathbf{y}(\mathbf{x}(t_f))$$

$$\mathbf{u}(0) = \pi^*(\mathbf{x}_0, \mathbf{g})$$

The **optimal goal-conditioned policy** π^* minimizes trajectory cost

$$\int_0^{t_f} L(\mathbf{x}(t), \mathbf{u}(t)) dt + L_f(\mathbf{x}(t_f), t_f)$$

subject to the constraints

$$\begin{aligned} \dot{\mathbf{x}}(t) &= \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)), \quad t \in [0, t_f], \\ \mathbf{x}(0) &= \mathbf{x}_0, \\ \mathbf{h}(\mathbf{x}(t), \mathbf{u}(t), t) &\leq \mathbf{0}, \quad t \in [0, t_f], \\ \mathbf{y}(\mathbf{x}(t_f)) &= \mathbf{g}, \\ t_{\min} &\leq t_f \leq t_{\max}, \end{aligned}$$

solving an optimal control problem.

How do we obtain a performant, fast approximation of π^* ?

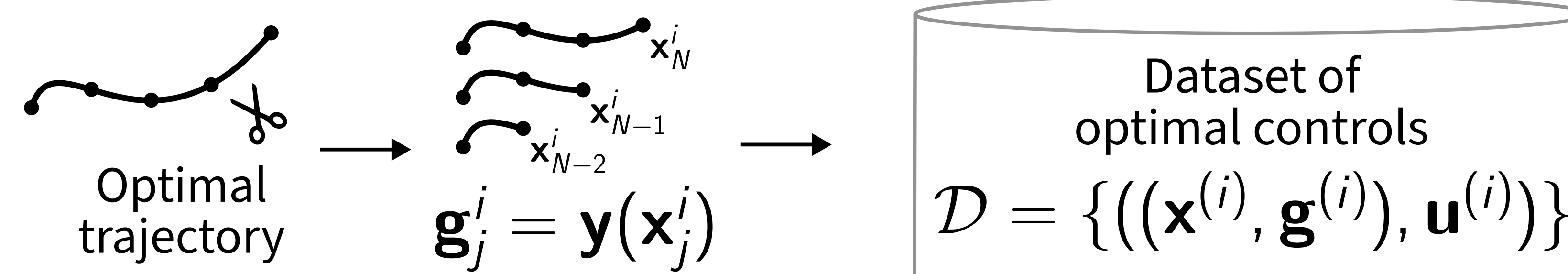
3 Method

1. Generate dataset of optimal trajectories:

$$\text{For } i \text{ from } 1 \text{ to } M: (\mathbf{x}_0^i, \mathbf{g}_i) \sim \mathcal{T}$$

Trajectory optimization with fast solver FATROP [1]
(minimize cost to reach $\mathbf{g}_i$ from $\mathbf{x}_0^i$)

Goal relabeling / Hindsight relabeling [2]



2. Train neural network policy π_θ by **behavior cloning**:

$$\min_{\theta} \sum_{((\mathbf{x}, \mathbf{g}), \mathbf{u}) \in \mathcal{D}} \|\pi_\theta(\mathbf{x}, \mathbf{g}) - \mathbf{u}\|_2^2$$

3. Evaluate closed-loop **success rate** and **near-optimality**:

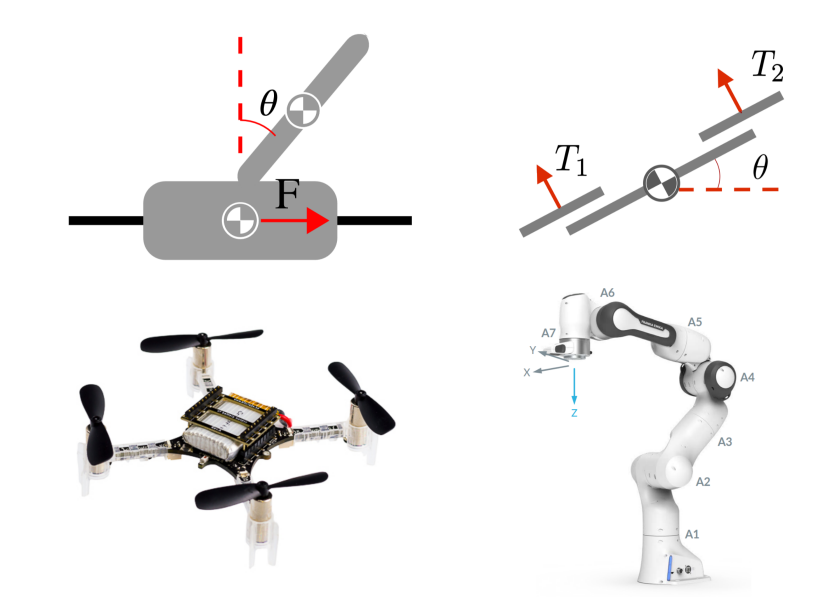
$$(\mathbf{x}_0^i, \mathbf{g}_i) \sim \mathcal{T} \xrightarrow{\text{simulate with } \pi_\theta} \begin{matrix} \text{Goal reached with} \\ \text{sufficient tolerance?} \\ \text{Trajectory cost } J \\ \text{Optimal cost } J^* \end{matrix} \rightarrow \delta_r = \frac{J - J^*}{J^*}$$

4 Experiments

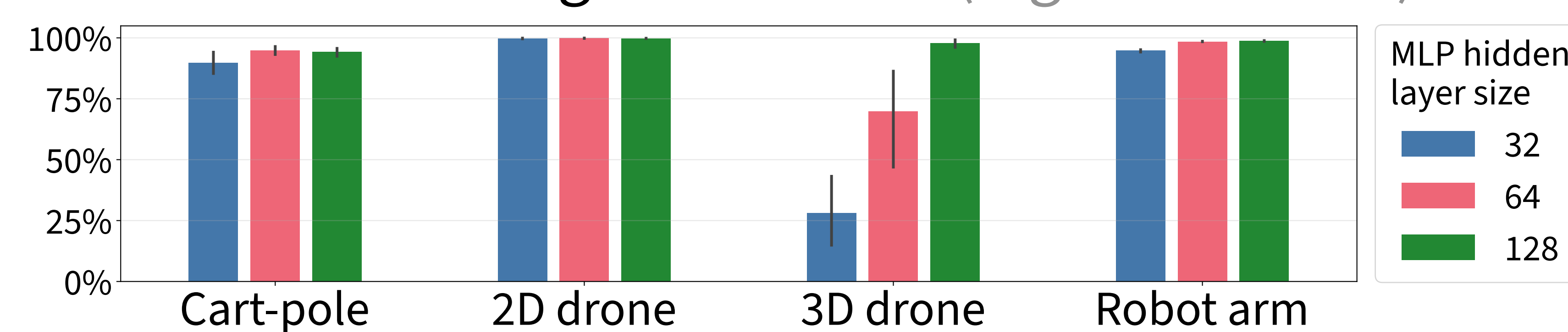
Continuous control tasks in simulation [3]:

cart-pole, 2D drone, 3D drone, robot arm.

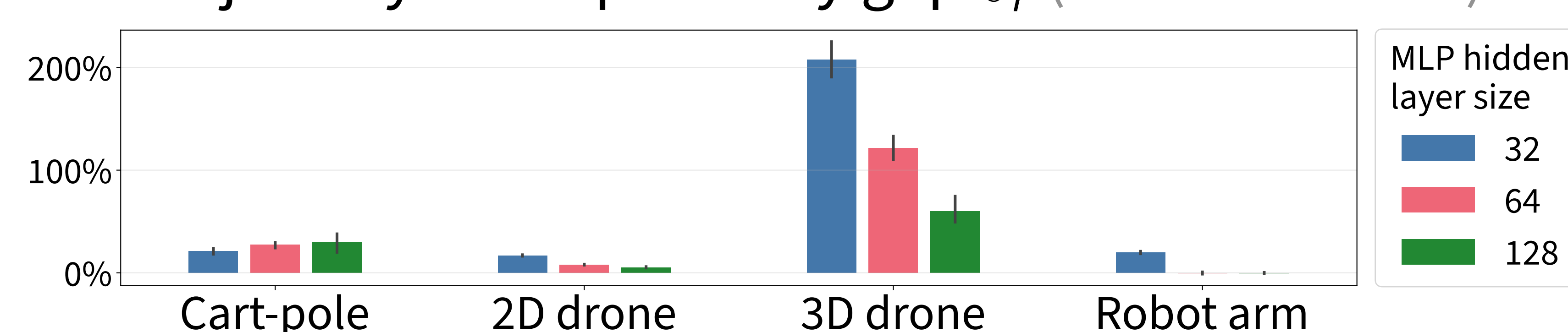
$$\text{Trajectory costs} = \alpha \int_0^{t_f} \|\mathbf{u}(t)\|_2^2 dt + (1 - \alpha) t_f$$



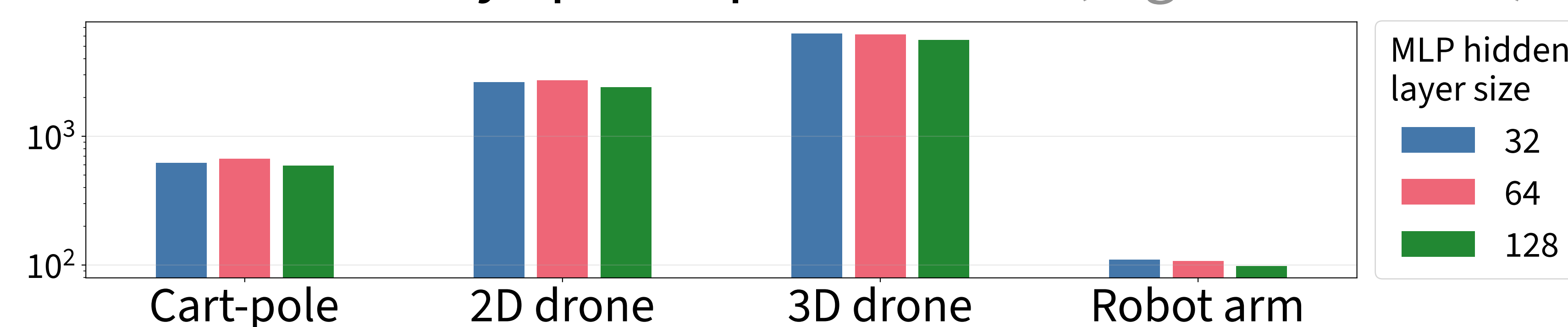
Goal reaching success rate (higher is better)



Trajectory cost optimality gap δ_r (lower is better)



Control latency speedup vs. FATROP (higher is better)

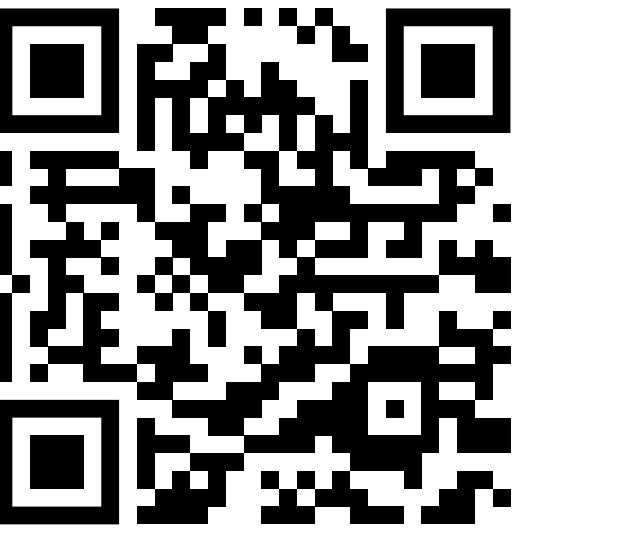


5 Analysis and discussion

GCImpOpt yields controllers with:

- **high success rates**,
 - **efficient control**,
 - **low control latency**.
- Dataset generation + training
in < 1 hour on a laptop.

Project website:
policy videos,
code,
data and models.



jongoiko.github.io/gcimopt

Conclusions:

- In general,

Lower regression error

$\neq$

Better closed-loop performance.

- 3D quadrotor is challenging:
high dimensionality, sensibility of dynamics.
- Although trajectory dataset can provide good state coverage, we still have *distribution shift* in behavior cloning [4]:

distr. of expert states/goals

$\neq$

distr. of states/goals visited by policy

6 Future work

How to deploy GCImpOpt policies confidently in real hardware?

- Handle plant-model mismatch and sensor/actuator noise.
- Ensure policy stability.

How to train policies for constrained near-optimal control? We only use

$$\mathbf{h}(\mathbf{x}(t), \mathbf{u}(t), t) \leq \mathbf{0}, \quad t \in [0, t_f]$$

for simple state/control bounds.
How can we incorporate e.g. obstacle avoidance?

How does GCImpOpt performance scale to higher-dimensional control tasks?

References

- [1] Vanroye, Lander, et al. "Fatrop: A fast constrained optimal control problem solver for robot trajectory optimization and control." 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2023.
- [2] Andrychowicz, Marcin, et al. "Hindsight experience replay." Advances in neural information processing systems 30 (2017).
- [3] Yuan, Zhacong, et al. "Safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics." IEEE Robotics and Automation Letters 7.4 (2022): 11142-11149.
- [4] Ross, Stéphane, Geoffrey Gordon, and Drew Bagnell. "A reduction of imitation learning and structured prediction to no-regret online learning." Proceedings of the fourteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings, 2011.